



IN THIS **PROJECT**, I HAVE UTILIZED **SQL** **QUERIES** TO ANALYZE A **COMPREHENSIVE PIZZA SALES DATASET**



 **GitHub**

github.com/Takee729

- DATA CLEANING AND IMPORTING
- TABLE JOINS
- AGGREGATE FUNCTIONS



RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED.



QUERY

```
3 • select count(order_id) as total_orders from orders;
```

OUTPUT

Result Grid	
	total_orders
▶	21350

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

QUERY

```
3 • SELECT
4   ROUND(SUM(orders_details.quantity * pizzas.price),
5          2) AS total_sales
6 FROM
7   orders_details
8   JOIN
9   pizzas ON pizzas.pizza_id = orders_details.pizza_id
10
```

OUTPUT

Result Grid 	
	total_sales
	817860.05

IDENTIFY THE HIGHEST-PRICED PIZZA.

QUERY

```
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

OUTPUT

Result Grid			Filter Rows
	name	price	
▶	The Greek Pizza	35.95	



IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED.

QUERY

```
SELECT
    pizzas.size,
    COUNT(orders_details.order_details_id) AS order_count
FROM
    pizzas
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```

OUTPUT

Result Grid			Filter
	size	order_count	
▶	L	18526	
	M	15385	
	S	14137	
	XL	544	
	XXL	28	

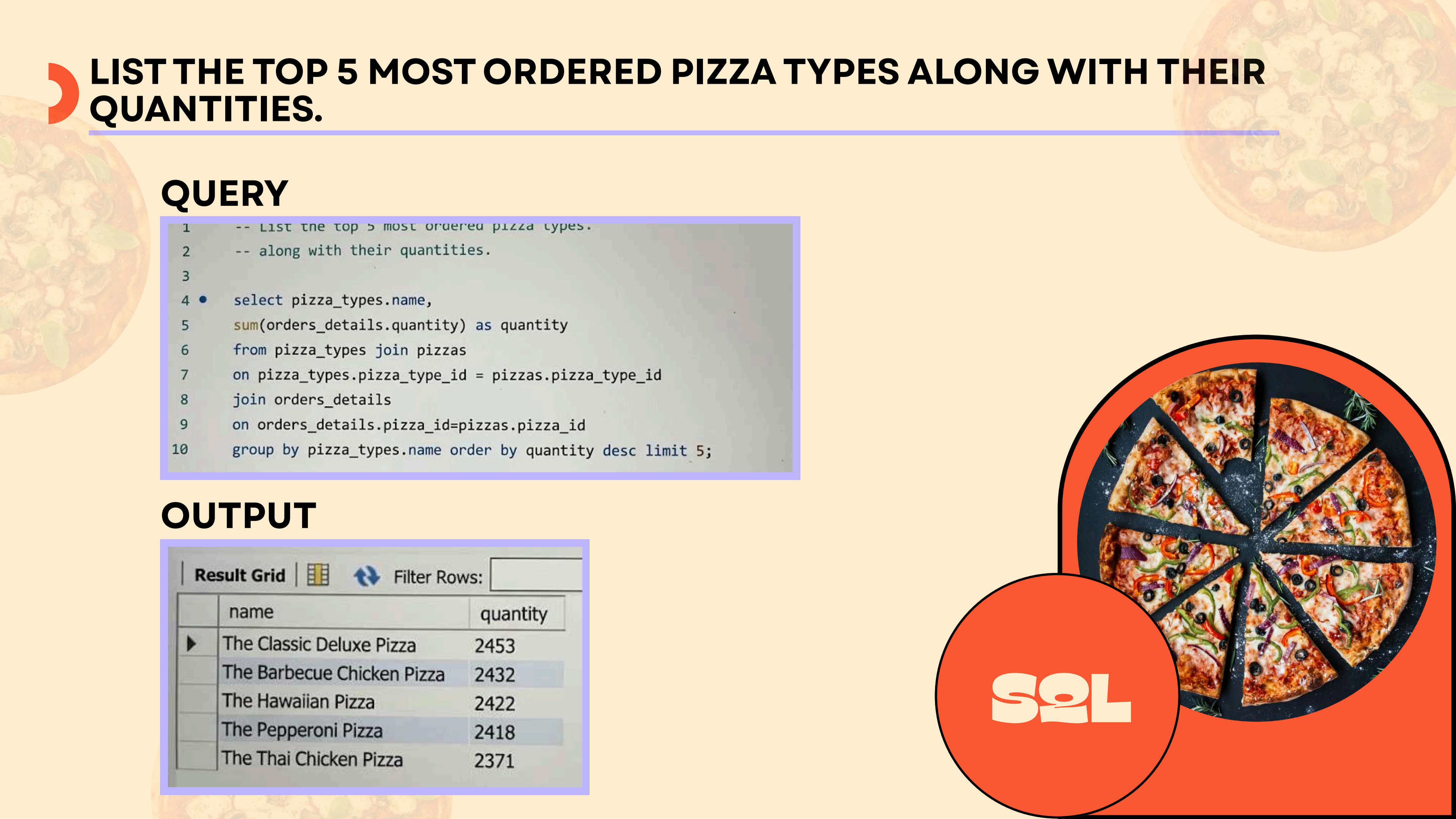
LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES.

QUERY

```
1  -- List the top 5 most ordered pizza types.
2  -- along with their quantities.
3
4  • select pizza_types.name,
5     sum(orders_details.quantity) as quantity
6  from pizza_types join pizzas
7   on pizza_types.pizza_type_id = pizzas.pizza_type_id
8   join orders_details
9   on orders_details.pizza_id=pizzas.pizza_id
10  group by pizza_types.name order by quantity desc limit 5;
```

OUTPUT

Result Grid			Filter Rows:
	name	quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	



SQL

 JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

QUERY

```
4 select pizza_types.category,  
5    sum(orders_details.quantity) as quantity  
6 from pizza_types join pizzas  
7  on pizza_types.pizza_type_id = pizzas.pizza_type_id  
8  join orders_details  
9  on orders_details.pizza_id = pizzas.pizza_id  
10 group by pizza_types.category order by quantity desc;
```

OUTPUT



	category	quantity
▶	Classic	14888
	Supreme	11987
	Veggie	11649
	Chicken	11050

DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

QUERY

```
1  -- Determine the distribution of orders by hour of the day.
2
3  • select hour(order_time) as hour, count(order_id) as order_count from orders
4    group by hour(order_time);
```

OUTPUT

Result Grid			Filter
	hour	order_count	
▶	11	1231	
	12	2520	
	13	2455	
	14	1472	
	15	1468	
	16	1920	
	17	2336	
	18	2399	
	19	2009	
	20	1642	
	21	1198	
	22	663	
	23	28	
	10	8	
	9	1	



JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

QUERY

```
1  -- Join relevant tables to find the category-wise
2  -- distribution of pizzas.
3
4  • select category, count(name) from pizza_types
5     group by category;
6
```

OUTPUT

Result Grid		Filter Ro
	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9



GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

QUERY

```
4 • select round(avg(quantity),0) as avg_pizza_ordered_per_day from
5   (select orders.order_date, sum(orders_details.quantity) as quantity
6    from orders join orders_details
7    on orders.order_id = orders_details.order_id
8    group by orders.order_date) as order_quantity;
```

OUTPUT

Result Grid		Filter R
	round(avg(quantity),0)	
▶	138	



DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

QUERY

```
3 • select pizza_types.name,  
4    sum(orders_details.quantity * pizzas.price) as revenue  
5    from pizza_types join pizzas  
6    on pizzas.pizza_type_id = pizza_types.pizza_type_id  
7    join orders_details  
8    on orders_details.pizza_id = pizzas.pizza_id  
9    group by pizza_types.name order by revenue desc limit 3;
```

OUTPUT

Result Grid			Filter Rows:
	name	revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	



CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE.

QUERY

```
4 • select pizza_types.category,  
5 round(sum(orders_details.quantity*pizzas.price) / (SELECT  
6 ROUND(SUM(orders_details.quantity * pizzas.price),  
7 2) AS total_sales  
8 FROM  
9 orders_details  
10 JOIN  
11 pizzas ON pizzas.pizza_id = orders_details.pizza_id)*100,2) as revenue  
12 from pizza_types join pizzas  
13 on pizza_types.pizza_type_id = pizzas.pizza_type_id  
14 join orders_details  
15 on orders_details.pizza_id = pizzas.pizza_id  
16 group by pizza_types.category order by revenue desc;
```



Result Grid			Filter
	category	revenue	
	Classic	26.91	
	Supreme	25.46	
	Chicken	23.96	
	Veggie	23.68	

OUTPUT

ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME.

QUERY

```
3 • select order_date,  
4    sum(revenue) over(order by order_date) as cum_revenue  
5    from  
6    (select orders.order_date,  
7     sum(orders_details.quantity * pizzas.price) as revenue  
8     from orders_details join pizzas  
9     on orders_details.pizza_id = pizzas.pizza_id  
10    join orders  
11    on orders.order_id = orders_details.order_id  
12    group by orders.order_date) as sales;
```

OUTPUT

Result Grid			Filter Rows:
	order_date	cum_revenue	
▶	2015-01-01	2713.85000000000004	
	2015-01-02	5445.75	
	2015-01-03	8108.15	
	2015-01-04	9863.6	
	2015-01-05	11929.55	
	2015-01-06	14358.5	
	2015-01-07	16560.7	
	2015-01-08	19399.05	
	2015-01-09	21526.4	
	2015-01-10	23990.3500000000002	
	2015-01-11	25862.65	

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

QUERY

```
4 • select name, revenue from
5
6 (select category, name, revenue,
7  rank() over(partition by category order by revenue desc) as rn
8  from
9  (select pizza_types.category, pizza_types.name,
10   sum((orders_details.quantity) * pizzas.price) as revenue
11   from pizza_types join pizzas
12   on pizza_types.pizza_type_id = pizzas.pizza_type_id
13   join orders_details
14   on orders_details.pizza_id = pizzas.pizza_id
15   group by pizza_types.category, pizza_types.name) as a) as b
16 where rn <= 3;
17 ;
```

OUTPUT

Result Grid			Filter Rows:	E
	name	revenue		
▶	The Thai Chicken Pizza	43434.25		
	The Barbecue Chicken Pizza	42768		
	The California Chicken Pizza	41409.5		
	The Classic Deluxe Pizza	38180.5		
	The Hawaiian Pizza	32273.25		
	The Pepperoni Pizza	30161.75		
	The Spicy Italian Pizza	34831.25		
	The Italian Supreme Pizza	33476.75		
	The Sicilian Pizza	30940.5		
	The Four Cheese Pizza	32265.700000000065		
	The Mexicana Pizza	26780.75		
	The Five Cheese Pizza	26066.5		